

INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Trabajo Terminal I · Registro 2027-A178 · Ingeniería en Sistemas Computacionales

Desarrollo de un General Game Player (GGP) utilizando un lenguaje de descripción de reglas para juegos de información perfecta

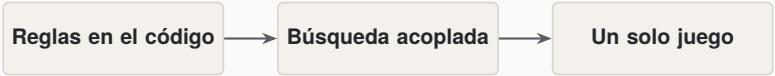
PRESENTAN
Sánchez Catarino Edgar
Linares Villegas Gustavo

DIRECTOR
Dr. en C. Enrique Escalante Notario

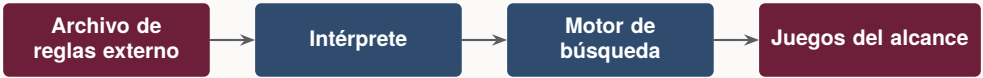
IDEA CENTRAL

Desacoplar las reglas del motor

Motor especializado



General Game Player



Al cambiar de juego se carga otra descripción y se construye su modelo; el núcleo conserva su código.

Fuente: Cap. 1, §1.2 y §1.5; Cap. 3, §3.3 [1], [2].

PROBLEMA

Un agente por juego: la hiperespecialización

1959 · Damas

Samuel

Minimax con evaluación heurística y mejora con la experiencia [3].

1997 · Ajedrez

Deep Blue

Búsqueda alfa-beta masivamente paralela con una evaluación hecha para ajedrez [4].

2018 · Ajedrez, Shogi y Go

AlphaZero

Un mismo algoritmo, pero las reglas de cada juego siguen programadas en su simulador [5].

Estos sistemas utilizan reglas y representaciones definidas para sus juegos.

Shannon estimó del orden de 10^{43} configuraciones de piezas y 10^{120} variantes bajo supuestos de ramificación y duración. Son estimaciones distintas, no un censo de posiciones legales [6].

Fuente: Cap. 1, §1.1–§1.2; Cap. 2, §2.1.

Un núcleo, varios juegos

OBJETIVO GENERAL

Un GGP para juegos de información perfecta que interprete las reglas desde un lenguaje de descripción, sin modificar el código de su motor base.

1. Diseñar la sintaxis y la semántica del lenguaje de reglas.
2. Desarrollar el intérprete y el generador de estados.
3. Implementar el motor de decisión del agente.
4. Validar la generalización con Gato y Ajedrez, de modo que el cambio de comportamiento dependa solo del archivo.
5. Evaluar el desempeño táctico en Ajedrez mediante partidas contra un motor con ELO configurable.

ENFOQUE DE TT1

Revisión, planeación, análisis y diseño. Los cinco productos del protocolo corresponden al proyecto completo; esta etapa fija las bases documentales para construirlos y validarlos.

Fuente: Cap. 1, §1.4 y §1.6.5 [7].

ALCANCE

Cuatro condiciones que delimitan los juegos

Información perfecta
Todos ven el estado completo y el historial en todo momento.

Discretos
Estados y acciones finitos y enumerables.

Deterministas
Sin azar: cada acción produce un único estado sucesor.

Por turnos
Los jugadores alternan de forma secuencial.

FUERA DE ALCANCE

Competir con Stockfish · replicar AlphaZero o Student of Games · juegos con azar o información oculta.

Casos de estudio: el **Gato**, pequeño y revisable por completo, y el **Ajedrez**, en el otro extremo de complejidad. La información perfecta no implica por sí sola determinismo: la exclusión del azar es un supuesto adicional.

Fuente: Cap. 1, §1.5 [5], [8].

General Game Playing: de heurísticas a simulación

2005–2006
Heurísticas inferidas
ClunePlayer y Fluxplayer: Minimax con evaluación derivada de las reglas [9], [10].

2007–2012
Monte Carlo Tree Search
CadiaPlayer: UCT y simulaciones aleatorias, sin heurística de dominio [11].

2014–2015
Competencia GGP
Sancho: agente ganador de la competencia internacional de GGP en 2014 [12].

2018–hoy
Otros lenguajes y aprendizaje
Ludii, RBG y aprendizaje profundo en GGP [13], [14], [15].

GGP nace en 2005 con Stanford y la AAAI: agentes que reciben la descripción formal de un juego desconocido y lo juegan sin cambios en su código [1].

Fuente: Cap. 2, §2.2–§2.6.

Un juego es un sistema de transición de estados

$$\Gamma = \langle R,\ S,\ s_0,\ \{A_i\},\ \{L_i\},\ T,\ F,\ \{G_i\} \rangle$$

- R Roles o jugadores.
- S Estados, como conjuntos de hechos (*fluents*).
- s_0 Estado inicial.
- A_i Acciones de cada rol.
- L_i Legalidad: $L_i : S \to \mathcal{P}(A_i)$.
- T Transición determinista: $T : S \times \mathbf{A} \to S$.
- F Estados terminales.
- G_i Utilidad: $G_i : S \to [0, 100]$.

Interfaz hacia el motor: estado inicial, acciones legales, aplicar acción, *¿es terminal?* y utilidad. Un juego bien formado cumple *jugabilidad* (todo rol tiene una acción legal en estados no terminales) y *terminación* garantizada. Los turnos alternados usan la acción nula `noop`.

Fuente: Cap. 3, §3.4 [1], [2], [16].

Reglas externas: GDL, subconjunto o DSL

EJEMPLO EN GDL: GATO

```
; legalidad
(<= (legal ?p (mark ?r ?c))
    (true (control ?p))
    (true (cell ?r ?c b)))

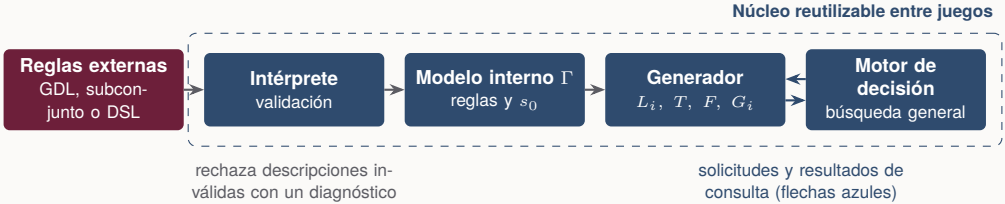
; alternancia de turno
(<= (next (control o))
    (true (control x)))
(<= (next (control x))
    (true (control o)))
```

Las reglas se *declaran*: el motor deduce las consecuencias.

GDL completo	Base académica y semántica lógica general; puede resultar pesado para un prototipo acotado.
Subconjunto	Puede simplificar la implementación; exige delimitar y justificar qué se admite.
DSL propio	Se concentra en lo que el proyecto necesita y puede ser más legible (como Ludii); debe justificarse formalmente.

Fuente: Cap. 3, §3.5–§3.6 [16], [17].

El motor de decisión no conoce ningún juego



Por turno: entran el estado s, el rol y un presupuesto. Si $s \in F$, se reportan utilidades; si no, el motor explora sucesores hipotéticos y elige una acción legal.

Cambiar de juego: se carga otra descripción y se construye su modelo interno. El núcleo conserva su código. La arquitectura es una propuesta conceptual.

Fuente: Cap. 5, §5.1 (Figura 5.1) y Cap. 3, §3.7.

De una descripción de reglas a un juego formal

Análisis de GDL

- Sintaxis y predicados del lenguaje.
- Restricciones de validez y semántica.
- Ejemplos de la resta y del Gato.
- Descripciones defectuosas y requisitos para el intérprete.

Del archivo al modelo

- Identificar símbolos y estructura de las reglas.
- Comprobar restricciones de la descripción.
- Asociar las reglas válidas al modelo Γ.
- Consultar legalidad, transiciones y resultados.

Fuente: Cap. 4, análisis del lenguaje de descripción de juegos.

¿Cómo comprobaremos que funciona?

Aspecto	Procedimiento	Indicador que se reportará
Corrección funcional	Comparar acciones legales y sucesores con una referencia; verificar terminalidad y utilidades.	Porcentajes separados para legalidad, transición, terminalidad y utilidad.
Calidad de decisión	Gato: comparar con soluciones óptimas. Ajedrez: primera jugada aceptada; acciones inválidas y tiempos agotados son fallos.	Porcentaje de aciertos por juego.
Costo de búsqueda	Medir cada decisión durante las pruebas.	Tiempo (mediana y p95), nodos expandidos y profundidad en medias jugadas; simulaciones MCTS aparte.
Generalidad operativa	Ejecutar ambos juegos con las mismas versiones del intérprete y del motor.	Versiones y componentes modificados.

Los indicadores de acierto usan $P_k = 100C_k/N_k$, con denominadores separados. Es una propuesta de evaluación; todavía no hay resultados.

Fuente: Cap. 1, §1.6, Tabla 1.2. Las muestras se presentan en la evaluación propuesta.

Prototipos de interfaz aprobados



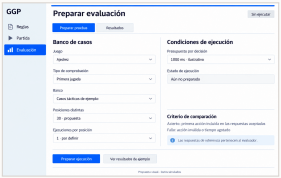
Carga y validación de reglas



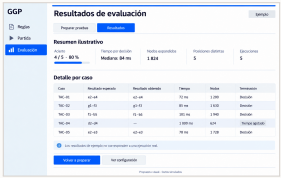
Partida de Gato



Partida de Ajedrez



Preparación de la evaluación



Reporte de resultados

PROTOTIPO VISUAL Datos simulados: no hay interfaz, intérprete ni agente implementados.

Fuente: Cap. 5, §5.2 (Figuras 5.2–5.6).

VALIDACIÓN PLANEADA

Evaluación propuesta para Gato y Ajedrez

Gato PROPUESTA caso funcional completo

- Revisión exhaustiva de estados alcanzables, contrastada con una referencia independiente.
- Cargar reglas, generar solo movimientos legales y jugar partidas completas con victoria y empate.
- Coincidencia del 100 % en los casos funcionales (propuesta).

Ajedrez

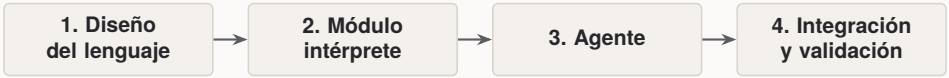
- Banco piloto exploratorio: 60 posiciones funcionales en seis familias.
- 30 problemas tácticos de primera jugada [18].
- python-chess como referencia de pruebas [19].
- Partidas contra un motor con ELO configurable, conforme al objetivo específico.

Los bancos de 60 posiciones funcionales y 30 problemas tácticos son exploratorios; complementarían las partidas contra motor previstas en el objetivo.

Fuente: Cap. 1, §1.6 (Tablas 1.2 y 1.3) [7], [20].

Etapas del desarrollo

Cuatro etapas del protocolo



Enfoque iterativo e incremental: cada ciclo produce un entregable revisable y las observaciones orientan el siguiente.

Cronograma del protocolo: actividades de agosto a junio, distribuidas entre los dos integrantes (detalle en el respaldo C).

Fuente: Cap. 1, §1.6; Cap. 9 [7].

APORTACIÓN

Una arquitectura desacoplada: reglas externas, motor general

Un núcleo de decisión sin conocimiento de ningún juego.

Un lenguaje de reglas con semántica explícita.

Al cambiar de juego, se carga otra descripción y se conserva el núcleo.

Gracias · Preguntas

Sánchez Catarino Edgar · Linares Villegas Gustavo
Director: Dr. en C. Enrique Escalante Notario

RESPALDO A

Predicados reservados de GDL

Predicado	Componente de Γ
`(role ?r)`	Roles R
`(init ?f)`	Estado inicial s_0
`(true ?f)`	Fluent verdadero en s
`(does ?r ?a)`	Acción a del rol r
`(legal ?r ?a)`	Legalidad $L_i(s)$
`(next ?f)`	Sucesor $T(s, \mathbf{a})$
`terminal`	Estados terminales F
`(goal ?r ?v)`	Utilidad $G_i(s)$

```
(role x)
(role o)
(init (cell 1 1 b))
; ... las demas casillas
(init (control x))
(<= (legal ?p noop)
    (role ?p)
    (not (true (control ?p))))
```

GDL es declarativo (Datalog con negación estratificada), independiente del dominio y desacopla la física del juego de los algoritmos de búsqueda.

Fuente: Cap. 3, §3.5 [2], [16].

Minimax, alfa-beta y MCTS con UCT

Minimax y poda alfa-beta

$$V(s) = \begin{cases} G_1(s) & s \in F \\ \max_{a \in L_1(s)} V(T(s,(a,\text{noop}))) & \text{turno de } r_1 \\ \min_{a \in L_2(s)} V(T(s,(\text{noop},a))) & \text{turno de } r_2 \end{cases}$$

Alfa-beta no cambia la decisión: poda ramas que no pueden mejorar el resultado conocido. Con profundidad limitada se requiere una evaluación general.

MCTS y selección UCT

Ciclo: selección, expansión, simulación y retropropagación.

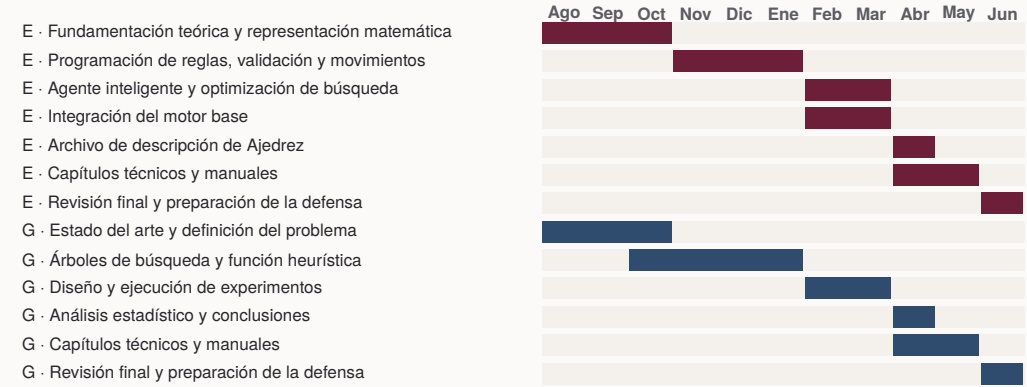
$$v^* = \arg\max_{v'} \left[\frac{Q_{p(v)}(v')}{N(v')} + C\sqrt{\frac{\ln N(v)}{N(v')}} \right]$$

Primero se seleccionan hijos sin visitas. $Q_{p(v)}$ acumula recompensas normalizadas del jugador activo.

Equilibra explotación y exploración sin heurísticas propias de un juego.

Fuente: Cap. 3, §3.9–§3.13 [21], [22], [23].

Cronograma del protocolo, de agosto a junio



E = Sánchez Catarino Edgar; G = Linares Villegas Gustavo. Las barras indican el periodo programado. Nombres abreviados.

Fuente: Cap. 9 [7].

Referencias citadas I

[1] M. Genesereth, N. Love y B. Pell, «General Game Playing: Overview of the AAAI Competition,» *AI Magazine*, vol. 26, n.º 2, págs. 62-72, 2005. dirección: https://logic.stanford.edu/publications/genesereth/aaai.pdf

[2] M. Genesereth y M. Thielscher, *General Game Playing*. Morgan & Claypool Publishers, 2014. dirección: https://logic.stanford.edu/ggp/chapters/cover.html

[3] A. L. Samuel, «Some Studies in Machine Learning Using the Game of Checkers,» *IBM Journal of Research and Development*, vol. 3, n.º 3, págs. 210-229, 1959. DOI: 10.1147/rd.33.0210

[4] M. Campbell, A. J. Hoane y F.-h. Hsu, «Deep Blue,» *Artificial Intelligence*, vol. 134, n.º 1–2, págs. 57-83, 2002. DOI: 10.1016/S0004-3702(01)00129-1

[5] D. Silver et al., «A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go Through Self-Play,» *Science*, vol. 362, n.º 6419, págs. 1140-1144, 2018. DOI: 10.1126/science.aar6404

[6] C. E. Shannon, «Programming a Computer for Playing Chess,» *Philosophical Magazine*, vol. 41, n.º 314, págs. 256-275, 1950. DOI: 10.1080/14786445008521796

[7] E. Sánchez Catarino y G. Linares Villegas, «Desarrollo de un General Game Player (GGP) utilizando un lenguaje de descripción de reglas para juegos de información perfecta,» Protocolo de Trabajo Terminal 2027-A178, Escuela Superior de Cómputo, Instituto Politécnico Nacional. Documento de trabajo proporcionado por los autores.

[8] M. Schmid et al., «Student of Games: A unified learning algorithm for both perfect and imperfect information games,» *Science Advances*, vol. 9, n.º 46, eadg3256, 2023. DOI: 10.1126/sciadv.adg3256

Referencias citadas II

[9] J. Clune, «Heuristic Evaluation Methods for General Game Playing,» en *Proceedings of the Twenty-Second AAAI Conference on Artificial Intelligence*, 2007, págs. 1133-1139.

[10] S. Schiffel y M. Thielscher, «Fluxplayer: A Successful General Game Player,» en *Proceedings of the Twenty-Second AAAI Conference on Artificial Intelligence*, 2007, págs. 1191-1196.

[11] H. Finnsson e Y. Björnsson, «Simulation-Based Approach to General Game Playing,» en *Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence*, 2008, págs. 259-264.

[12] S. Draper y A. Rose, «Sancho,» visitado 25 de oct. de 2026. dirección: https://sanchoggp.github.io/sancho-ggp/

[13] É. Piette et al., «Ludii – The Ludemic General Game System,» en *ECAI 2020*, 2020, págs. 411-418. DOI: 10.3233/FAIA200120 dirección: https://arxiv.org/abs/1905.05013

[14] J. Kowalski et al., «Regular Boardgames,» en *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence*, 2019, págs. 1699-1706. DOI: 10.1609/aaai.v33i01.33011699

[15] A. Goldwaser y M. Thielscher, «Deep Reinforcement Learning for General Game Playing,» en *Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence*, 2020, págs. 1701-1708. DOI: 10.1609/aaai.v34i02.5533

[16] N. Love et al., «General Game Playing: Game Description Language Specification,» Stanford Logic Group, inf. téc. LG-2006-01, 2008. dirección: https://logic.stanford.edu/ggp/

[17] D. J. N. J. Soemers et al., «The Ludii Game Description Language is Universal,» en *Proceedings of the 2024 IEEE Conference on Games (CoG)*, 2024, págs. 1-8. DOI: 10.1109/CoG60054.2024.10645550

Referencias citadas III

[18] Lichess, «Lichess open database: Puzzles,» visitado 17 de oct. de 2026. dirección: https://database.lichess.org/#puzzles

[19] N. Fiekas, «Core — python-chess documentation,» visitado 17 de oct. de 2026. dirección: https://python-chess.readthedocs.io/en/latest/core.html

[20] «Mejora académica del protocolo de Trabajo Terminal: TT 2027-A178,» Documento de recomendaciones proporcionado para el proyecto. No sustituye formalmente al protocolo aprobado.

[21] C. B. Browne et al., «A Survey of Monte Carlo Tree Search Methods,» *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, n.º 1, págs. 1-43, 2012. DOI: 10.1109/TCIAIG.2012.2186810

[22] L. Kocsis y C. Szepesvári, «Bandit Based Monte-Carlo Planning,» en *Machine Learning: ECML 2006*, ép. Lecture Notes in Computer Science, vol. 4212, Springer, 2006, págs. 282-293. DOI: 10.1007/11871842_29

[23] S. Russell y P. Norvig, *Artificial Intelligence: A Modern Approach*, 3.ª ed. Prentice Hall, 2010.